

Pricing Studio Platform

Solution Architecture & End-to-End Design

I designed this platform as a governed pricing decision system - starting with the domain and database model, then building deterministic pricing and profitability services, surrounding them with policy and approval workflow, and finally integrating enterprise data, deployment controls and AI-assisted pricing.

1. Design Goals and Principles

The flow is designed around a single principle: pricing must be fast enough for Sales to use directly, while remaining reproducible, profitable, controlled and auditable for Product, Finance, Risk and Compliance.

- **Self-service with governance:** Sales creates and compares scenarios; control functions retain policy and approval authority.
- **Configuration over code:** fee structures, tiers, discounts, bundles and thresholds are versioned configuration.
- **Deterministic calculations:** every result can be reproduced from stored inputs, rule/model version and cost assumptions.
- **Scenario-first:** users compare commercial alternatives before submitting a deal for approval.
- **Separation of concerns:** pricing calculation, profitability, policy and workflow are independent capabilities.
- **Human-in-the-loop AI:** AI recommends and explains; authorized users make the commercial decision.

2. End-to-End Solution Flow

Stage	Designed flow	Primary outcome
1. Deal Setup	Client -> products -> volumes -> terms -> currency -> commercial assumptions	Canonical deal context
2. Fee Design	Select fee types, tiers, minimums, discounts and bundles	Configurable commercial model
3. Scenario Calculation	Run base, discount, bundle, acquisition and AI-assisted scenarios	Comparable pricing alternatives
4. Profitability	Calculate revenue, cost-to-serve, profit, margin and relationship economics	Transparent P&L;
5. Policy Check	Evaluate margin floors, discount authority, risk and regulatory guardrails	Controlled exceptions
6. Approval Workflow	Route to Finance, Product, Risk or senior authority based on policy	Governed decision
7. Finalization	Freeze the approved deal/pricing version and publish lifecycle events	Auditable approved price
8. Analytics / AI	Feed outcomes to analytics and ML for future recommendations	Continuous decision improvement

3. High-Level System Architecture

Layer	Components	Design responsibility
Experience	React Pricing Studio	Deal Workspace, Fee Designer, Scenario Builder, Profitability Dashboard, Approval Center, AI Insights

Layer	Components	Design responsibility
API / Orchestration	Spring Boot API layer	Authentication, authorization, validation, API versioning and domain orchestration
Domain Services	Pricing, Profitability, Policy/Risk, Approval, Client, Reference Data, Audit	Explicit business boundaries and independently testable capabilities
Workflow	BPMN / Flowable-style engine	Multi-level approvals, timers, escalation, notifications and exception paths
Events	Kafka	Lifecycle events, decoupled integrations, analytics feeds and asynchronous processing
Data	Oracle + analytics platform	Transactional system of record, versioned pricing configuration, audit and reporting
AI	ML / recommendation services	Price recommendation, win probability, churn risk, demand forecast and explanation

Implementation choice: I would begin with a modular Spring Boot platform with clean domain modules. I would extract independently deployable services only when scale, ownership, release cadence or resilience requirements justify the additional operational complexity.

4. Pricing Studio User Experience

Module	Designed capabilities
Deal Workspace	Client, products, volumes, terms, currency, assumptions, ownership and lifecycle.
Fee Designer	AUM, transaction, fixed and performance fees; minimums, tiers, discounts and bundles.
Scenario Builder	Base, discounted, bundled, acquisition and AI-recommended scenarios with side-by-side comparison.
Profitability Dashboard	Revenue, cost, profit, margin, cost-to-serve, ROI and relationship economics.
Approval Center	Pending approvals, threshold violations, comments, decisions, escalation and history.
AI Recommendations	Recommended range, expected margin, win probability, churn risk, confidence and explanation.

5. Logical Database and Domain Model

I designed the data model so every pricing decision is reproducible. A pricing result references the exact deal version, scenario, pricing model version, cost assumptions and approval state used to produce it.

Entity	Purpose / important fields
CLIENT	client_id, name, segment, region, risk rating, relationship attributes
DEAL	deal_id, client_id, deal type, currency, owner, status, version, effective dates
PRODUCT	product_id, product type, service type, pricing model reference
DEAL_PRODUCT	deal_id, product_id, volume, term, dates and pricing assumptions
PRICING_MODEL	model_id, product_id, version, effective/expiration dates, status, approved_by
PRICING_RULE	rule_id, model_id, fee type, method, tier, rate, min/max, condition/expression
SCENARIO	scenario_id, deal_id, type, assumptions, status, created_by
PRICING_RESULT	result_id, deal/scenario/model versions, revenue, cost, profit, margin, calculated_at
COST_MODEL	cost_id, product_id, cost type, unit economics, currency and assumptions

Entity	Purpose / Important fields
APPROVAL	approval_id, deal/version, step, approver role, status, comments, timestamp
AUDIT_EVENT	event_id, user, action, entity, before/after values, timestamp, correlation_id
AI_INSIGHT	insight_id, deal_id, recommendation, confidence, explanation, model/version

- **Immutable approved versions:** create a new version rather than overwriting approved pricing configuration.
- **Effective dating:** select the correct model/rule set for the deal's effective date.
- **Traceability:** persist input, configuration version, cost version, result and approval lineage.
- **Auditability:** capture who changed what, before/after values, reason and correlation ID.

6. Dynamic Pricing Engine

The pricing engine is deterministic and rules-driven. It calculates commercial economics; it does not decide whether an exception is authorized.

Step	Processing
1. Validate	Validate required deal inputs, product eligibility, reference data and data quality.
2. Resolve model	Select product pricing model and rule version using effective date and context.
3. Base fees	Calculate AUM, transaction, fixed, performance and service-specific fees.
4. Tiering	Apply volume bands, AUM tiers, transaction thresholds and progressive rates.
5. Discounts	Apply relationship, strategic, term or client-specific discounts.
6. Bundling	Evaluate multi-product package economics and cross-product adjustments.
7. Overrides	Apply authorized commercial overrides while retaining original/base economics.
8. Economics	Produce revenue and pass the result to profitability for cost, profit and margin.

Key design boundary: Pricing Engine = calculation. Policy/Risk = whether the calculated/overridden outcome is allowed. Approval Workflow = who has authority to accept the exception.

7. Scenario Builder and Profitability

Scenario	Purpose
Base	Approved standard pricing model.
Discount	Measure commercial discount impact.
Bundle	Evaluate total economics across multiple products.
Acquisition	Test aggressive pricing for strategic wins.
AI Recommended	Use ML recommendation as another scenario, never as an automatic approval.

Profitability flow: Revenue - direct cost - operating/technology/servicing cost = profit -> margin. The platform also calculates revenue/AUM, revenue/transaction, cost-to-serve, contribution margin, ROI and relationship value.

8. Policy, Risk and Approval Workflow

Condition	Example policy	Designed action
Margin below floor	Margin < defined product/client threshold	Require Finance approval

Condition	Example policy	Designed action
Commercial discount	Discount exceeds salesperson authority	Route to Product + Finance
Large exception	Discount exceeds senior threshold	Add Risk and senior approval
Regulatory constraint	Deal violates a regulatory/product guardrail	Block or require Compliance/Risk exception path
AI recommendation	Recommendation materially changes pricing	Human approval remains mandatory

The workflow retains maker/checker separation, approval authority, comments, rejection/change requests, timers, escalation, notifications and a complete decision history.

9. Event and Enterprise Integration Architecture

Event	Producer	Primary consumers
DealCreated	Deal Service	Pricing, analytics, audit
PricingCalculated	Pricing Service	Profitability, UI, analytics
ScenarioCreated	Scenario capability	Pricing, profitability, audit
PricingChanged	Pricing Service	Workflow, audit, analytics
ApprovalRequested	Approval Service	Notifications, work queue, audit
ApprovalCompleted	Workflow	Deal lifecycle, audit, downstream integrations
DealApproved	Workflow	CRM, Finance, reporting, submission
DealRejected	Workflow	Sales notification, audit, analytics

- Kafka events use correlation IDs, schema versioning and idempotent consumers.
- Failed asynchronous integrations use retry and dead-letter handling.
- A transactional outbox or equivalent pattern protects database/event consistency.
- CRM supplies client/deal context; Finance supplies cost/revenue data; reference-data services govern product and market attributes.

10. AI-Powered Pricing Target State

I place AI after the deterministic pricing, cost, workflow and historical-outcome foundations are stable. This makes the AI layer explainable and governable rather than embedding opaque logic into core calculations.

Inputs	AI / ML capabilities	Outputs
Won/lost deals, AUM, transaction volume, product mix, discount history, cost-to-serve, relationship attributes, market signals	Price/range recommendation, win model, churn model, demand forecast, feature attribution	Recommended range, win probability, expected revenue/margin, churn risk, confidence, explanation

- Persist model/version and feature snapshot with each recommendation.
- Display major drivers and confidence.
- Monitor drift and accuracy by product/client segment.
- Use deterministic fallback when AI is unavailable or confidence is insufficient.
- AI never bypasses policy, approval authority or regulatory controls.

11. AWS Deployment Architecture

Area	Designed deployment pattern
Edge	Route 53 -> CloudFront -> Application Load Balancer
Runtime	EKS with deployments/pods for pricing, profitability, policy, workflow and supporting services
Database	Enterprise-supported Oracle deployment as transactional system of record
Events	Managed/enterprise Kafka integration
Artifacts	S3 for appropriate documents/static assets; ECR for container images
Security	Enterprise SSO/OIDC, RBAC + ABAC, Secrets Manager, KMS, private networking and least privilege
Observability	Centralized logs, CloudWatch metrics/alerts, distributed tracing and business telemetry
CI/CD	Git -> build/test -> quality/security scan -> image registry -> controlled EKS deployment
Resilience	Multi-AZ, health checks, autoscaling, retries, circuit breakers and disaster recovery

12. Delivery Sequence

Phase	Focus	Indicative duration	Outcome
1	Domain & Data Design	4-6 weeks	Products, fee models, pricing rules, approval policies, cost model and canonical data design
2	Pricing MVP	8-12 weeks	Deal workspace, fee configuration, deterministic pricing and scenario comparison
3	Governance	6-10 weeks	BPMN workflow, thresholds, approvals, audit, notifications and exceptions
4	Enterprise Integration	6-10 weeks	CRM, Finance, reference data, Kafka, reporting and analytics feeds
5	Advanced Profitability	4-6 weeks	Cost-to-serve, attribution, margin forecasting and relationship economics
6	AI Pricing	6-8 weeks	Recommendations, win/churn models, explainability and controlled feedback loop

13. Non-Functional Design

Area	Design requirement
Performance	Interactive scenario recalculation for normal deal sizes; isolate expensive calculations when required.
Consistency	Explicit currency, rounding and effective-date rules; deterministic financial outputs.
Traceability	Every result maps to input, deal version, pricing rule/model, cost model and code/config version.
Security	SSO, least privilege, RBAC/ABAC, data entitlements and approval authority.
Testing	Golden pricing cases, rule tests, contract tests, workflow tests and regression packs.
Release safety	Versioned APIs/configuration, backward compatibility and controlled rollout.
Observability	Technical telemetry plus business-level pricing/workflow metrics.

14. Design Rationale

- **I start with the domain and database design** so pricing is versioned, reproducible and traceable before service boundaries are finalized.
- **I separate deterministic pricing from profitability** so fee logic and financial economics can evolve independently.
- **I surround pricing with policy and workflow** so exceptions are visible, governed and auditable instead of hidden in calculation code.
- **I use Kafka for lifecycle integration** so core pricing transactions remain decoupled from CRM, Finance, analytics and downstream systems.
- **I introduce AI only after reliable data foundations exist** and keep human approval as the final control point.
- **I deploy with enterprise security and observability** so the design can move from a pricing MVP to a production-grade platform without changing its core domain model.

15. Target Outcome

The resulting flow provides faster self-service pricing, transparent profitability, controlled commercial exceptions, complete auditability, reusable/versioned pricing configuration and AI-assisted decision quality. The implementation sequence is deliberate: domain/database -> deterministic pricing -> scenarios/profitability -> policy/workflow -> enterprise integration -> deployment hardening -> AI augmentation.